DRIFTLESS — File-by-File Technical Codebase Analysis

This document provides a line-by-line and architectural breakdown of every meaningful source file in the DRIFTLESS repository.

---

1. Extension Layer Files

1.1. `extension/manifest.json`

- **File Type & Layer:** JSON Configuration · Extension Runtime Manifest
- **Responsibility:** Declares extension identity, Manifest V3 specification, permissions, assets, and service worker registration.
- **Key Fields & Constants:**
- `manifest_version: 3` : Enforces Chrome's modern service worker architecture.
- `permissions` :
  - `tabs` : Allows querying active browser tabs and reading tab URLs for domain tracking.
  - `storage` : Enables asynchronous key-value persistence via `chrome.storage.local` .
  - `alarms` : Grants access to scheduled background intervals for 60s periodic sync and midnight reset.
  - `notifications` : Allows spawning desktop system notification cards for nudges and alerts.
  - `idle` : Enables user inactivity detection via `chrome.idle` .
  - `declarativeNetRequest` , `declarativeNetRequestWithHostAccess` , `declarativeNetRequestFeedback` : Grants permission to dynamically insert network redirect rules without requiring invasive broad content scripts.
- `host_permissions: ["<all_urls>"]` : Permits network interception and domain inspection across all HTTP/HTTPS destinations.
- `web_accessible_resources` : Exposes `blocked.html` , `theme.css` , and `icons/*` to web pages for redirects.
- `background.service_worker: "background.js"` : Designates the service worker script.
- **Dependencies:** Loads `background.js` , `popup.html` , `theme.css` , and icon binaries.

1.2. `extension/background.js`

- **File Type & Layer:** JavaScript (ES2022) · Service Worker / Background Controller
- **Responsibility:** Primary client-side tracking engine. Measures active tab dwell duration, detects window focus and user idle states, evaluates 3-tier escalation thresholds, executes dynamic redirects via `declarativeNetRequest`, and flushes periodic time deltas to the Django REST API.
- **Key Constants & Variables:**
- `ALARM_PERIODIC_SYNC = 'driftless_periodic_sync'`: Triggers every 60 seconds.
- `ALARM_MIDNIGHT_RESET = 'driftless_midnight_reset'`: Daily midnight cleanup alarm.
- `ALARM_FOCUS_TIMER_END = 'driftless_focus_timer_end'`: Fires upon focus block completion.
- `FOCUS_RULE_BASE_ID = 100`: Base ID for focus mode DNR rules.
- `LIMIT_BLOCK_RULE_BASE_ID = 1000`: Base ID for limit violation DNR redirect rules.
- `currentTabDomain`, `currentTabStartTime`, `isWindowFocused`, `isUserActive`: In-memory tracking state.
- **Core Functions & Execution Flows:**
  1. `cleanDomain(url)`:
     - *Input:* Full URL string (`https://www.youtube.com/watch?v=xyz`).
     - *Processing:* Uses `new URL(url)`, verifies `http:` or `https:`, strips `www.`, and extracts lowercase hostname (`youtube.com`).
     - *Output:* Sanitized domain string or `null`.
  2. `commitActiveTime()`:
     - *Execution Flow:*
       1. Verifies if `currentTabDomain`, `currentTabStartTime`, `isWindowFocused`, and `isUserActive` are valid.
       2. Computes `elapsedSeconds = Math.floor((Date.now() - currentTabStartTime) / 1000)`.
       3. Updates `currentTabStartTime = Date.now()`.
       4. Reads `siteTimes` and `unsyncedDeltas` from `chrome.storage.local`.
       5. Increments seconds for `currentTabDomain` in both dictionaries.
       6. Saves back to storage and invokes `evaluateSiteLimits(domain, newTotalSeconds)`.
  3. `evaluateSiteLimits(domain, totalSeconds)`:
     - *Execution Flow (3-Tier Escalation):*
       1. Pulls `siteLimits`, `userTimeLimitMinutes`, and `escalationState` from storage.
       2. Computes `limitSeconds = limitMins * 60` and `gracePeriod = 300` (5 minutes).
       3. **Tier 1 (80% Nudge):** If `totalSeconds ≥ limitSeconds * 0.8` and not nudged: sets `state.nudged = true` and triggers `sendNudgeNotification()`.
       4. **Tier 2 (100% Alert):** If `totalSeconds ≥ limitSeconds` and not alerted: sets `state.alerted = true`, marks `streakAtRisk = true`, and calls `sendAlertNotification()`.
       5. **Tier 3 (100% + 5m Block):** If `totalSeconds ≥ limitSeconds + 300` and not blocked:

sets `state.blocked = true` and invokes `blockDomainViaDNR(domain)`.
4. `blockDomainViaDNR(domain)`:
    - *Processing:* Checks existing dynamic rules. If none exist for this domain, allocates a unique rule ID (`LIMIT_BLOCK_RULE_BASE_ID + random`), creates a dynamic redirect action pointing to `chrome.runtime.getURL('blocked.html?site=' + domain)`, and calls `chrome.declarativeNetRequest.updateDynamicRules()`. If the user is currently looking at that tab, immediately calls `chrome.tabs.update()` to redirect them.
5. `syncToBackend()`:
    - *Processing:* Reads `apiToken` and `unsyncedDeltas`. If tokens and positive deltas exist, builds a JSON array `[{ website_url, time_spent_seconds, date }]` and transmits via `POST ${serverUrl}/api/usage/`. Upon HTTP 200, clears `unsyncedDeltas` and updates `lastSyncedTime`. Follows up with `pullBackendConfig()` to refresh settings and limits.

- **Event Listeners:** `chrome.tabs.onActivated`, `chrome.tabs.onUpdated`, `chrome.windows.onFocusChanged`, `chrome.idle.onStateChanged`, `chrome.alarms.onAlarm`, `chrome.runtime.onMessage`.

1.3. `extension/popup.html` & `extension/popup.js`

- **File Type & Layer:** HTML5 / JavaScript · Extension User Interface
- **Responsibility:** Serves as the interactive extension drop-down menu. Displays real-time streak badges, connected server indicator, live focus timer buttons, per-site daily usage bars, domain limit addition form, API token input, theme toggle button, and reset confirmation modal.
- **Core Functions in `popup.js`:**
- `loadPopupState()`: Reads `chrome.storage.local` keys (`apiToken`, `streakDays`, `focusModeActive`, `siteTimes`, `siteLimits`) and populates the DOM.
- `renderSiteList(siteTimes, siteLimits)`: Merges all tracked and limited domains. Renders visual progress bars colored dynamically (green for normal, amber for 60%+, rose for 80%+ limit usage).
- `renderFocusTimerCountdown()`: Updates ticking countdown display `⏱ MM:SS` when a focus session is active.
- `startLiveTimerTicker()`: Sets a 1-second `setInterval` to keep the popup countdown clock synchronized.
- `setupEventListeners()`:
    - Focus Mode toggle emits `SET_FOCUS_MODE` and PATCHes `/api/settings/`.
    - Focus Timer preset buttons (25m, 45m, 60m) send `START_FOCUS_TIMER` message to background.
    - Add Limit button validates domain and limit minutes, updates `chrome.storage.local`, and POSTs to `/api/limits/`.

- Save Token button commits user token to `chrome.storage.local` and triggers an immediate sync.
- Reset Timers button displays custom confirmation modal before sending `RESET_ALL_TIMERS`.

1.4. `extension/blocked.html`

- **File Type & Layer:** HTML5 / Embedded CSS · Network Interception Landing Screen
- **Responsibility:** Replaces blocked tabs when daily allowances expire or when Focus Mode is active.
- **Design & Behavior:**
- Extracts the blocked domain from URL search query parameter `?site=...`.
- Displays a clean card with Calm Sky / Opal Noir theme tokens, warning icon, domain pill, and supportive streak preservation message.
- Includes a "Back to Productive Work →" button redirecting users to `https://github.com` and a link to the Driftless dashboard (`http://127.0.0.1:8000/dashboard/`).

1.5. `extension/theme.css`

- **File Type & Layer:** CSS3 Custom Properties · Extension Theme Tokens
- **Responsibility:** Declares extension-specific styling tokens for Light mode (`:root`, `[data-theme="light"]`) and Dark mode (`[data-theme="dark"]`).
- **Variables Declared:** `--bg`, `--card`, `--border`, `--ink`, `--ink-muted`, `--pri`, `--accent`, `--grad`, `--prod`, `--soc`, `--ent`.

---

2. Dashboard Backend Files

2.1. `dashboard/driftless/settings.py`

- **File Type & Layer:** Python Configuration · Django Core Settings
- **Responsibility:** Configures application settings, middleware pipeline, installed apps, database connection, static assets, and security parameters.
- **Key Configurations:**
- `INSTALLED_APPS`: Registers `django.contrib.*`, `rest_framework`, `rest_framework.authtoken`, `corsheaders`, and `core`.

- `MIDDLEWARE`: Positions `corsheaders.middleware.CorsMiddleware` first, followed by `WhiteNoiseMiddleware` for compressed static asset delivery.
- `DATABASES`: Configures `django.db.backends.sqlite3` targeting `BASE_DIR / 'db.sqlite3'`.
- `REST_FRAMEWORK`: Enforces `TokenAuthentication` and `SessionAuthentication`, default permission `IsAuthenticated`, and throttling tiers (`anon: 100/day`, `user: 1000/day`).
- `CORS_ALLOW_ALL_ORIGINS = True` & `CORS_ALLOWED_ORIGIN_REGEXES = [r'^chrome-extension://.*$']`: Allows extension fetch requests.
- `CSRF_TRUSTED_ORIGINS`: Lists development and tunneling domains (ngrok, cloudflare, loca.lt).

2.2. `dashboard/core/models.py`

- **File Type & Layer:** Python ORM · Data Schema & Domain Logic
- **Responsibility:** Defines domain classification algorithm, master categories, and relational models.
- **Key Functions & Models:**
  1. `clean_domain(domain: str) → str`: Normalizes domain strings by stripping protocols (`http://`, `https://`), `www.`, ports, query parameters, and trailing paths.
  2. `classify_domain(domain: str, user=None) → str`:
     - *Priority 1:* Checks if user has a custom `CategoryOverride` for this domain.
     - *Priority 2:* Evaluates user persona (e.g. Influencers categorize Instagram/YouTube as `Productivity`; Professionals categorize LinkedIn/Slack as `Productivity`).
     - *Priority 3:* Regex match on academic domains (`.edu`, `.ac.in`) -> `Productivity`.
     - *Priority 4:* Global `CATEGORY_MAP` dictionary lookup.
     - *Priority 5:* Fallback to `Other`.

     3. `UserProfile`: Extends Django's `User` with `phone` and `persona` (`student`, `professional`, `influencer`, `creator`, `other`).
     4. `CategoryOverride`: User-customized classification for a domain (`user`, `domain`, `category`). Unique together on `('user', 'domain')`.
     5. `SiteLimit`: Daily threshold per domain in minutes (`user`, `domain`, `daily_limit_minutes`, `enabled`).
     6. `FocusSession`: Discrete focus timer blocks (`user`, `duration_minutes`, `started_at`, `completed`).
     7. `UsageLog`: Accumulated dwell time per domain per calendar day (`user`, `website_url`, `time_spent_seconds`, `category`, `date`). Unique together on `('user', 'website_url', 'date')`. Indexed on `['user', 'date']`.
     8. `UserSettings`: High-level user flags (`focus_mode_active`, `daily_limit_minutes`, `focus_goal_minutes`, `streak_days`, `require_challenge`, `blocked_sites`, `allowed_sites`).

2.3. `dashboard/core/serializers.py`

- **File Type & Layer:** Python / DRF · Data Serialization & Validation
- **Responsibility:** Validates incoming JSON payloads and deserializes them into model instances.
- **Key Classes & Utilities:**
- `apply_persona_defaults(user, persona)`: Populates default site limits (e.g., student gets 30m Instagram, 45m YouTube, 20m Reddit, 40m Netflix).
- `RegisterSerializer`: Validates registration input (`username`, `email`, `phone`, `password`, `persona`). Converts spaces in usernames to underscores, validates phone formats, enforces unique constraints, creates `UserProfile` and `UserSettings`, and applies persona limits.
- `UsageLogSerializer`: Validates `time_spent_seconds ≥ 0` and `≤ 86400` (24h safety clamp), cleans domain.
- `SiteLimitSerializer`: Enforces `daily_limit_minutes` between 1 and 1440 minutes (24 hours).
- `CategoryOverrideSerializer`, `FocusSessionSerializer`, `UserSettingsSerializer`: Standard model serializers.

2.4. `dashboard/core/views.py`

- **File Type & Layer:** Python / DRF · Controller & Business Logic
- **Responsibility:** Handles template rendering, authentication endpoints, batch usage log ingestion, 7-day analytics aggregation, strict streak calculation, settings updates, and data sovereignty.
- **Key View Functions:**
- `auth_page(request)`: Renders `auth.html` or redirects authenticated sessions to `/dashboard/`.
- `dashboard_page(request)`: Authenticated view passing `api_token`, `username`, and `persona` into `dashboard.html`.
- `register_view(request)`: `POST /api/auth/register/` creates user, issues DRF token, logs in Django session.
- `login_view(request)`: `POST /api/auth/login/` resolves identifier (username, email, or normalized phone), authenticates credentials, issues token, creates session.
- `logout_view(request)` & `regenerate_token_view(request)`: Revoke or re-issue tokens.
- `usage_log_view(request)`: `POST /api/usage/` throttled at `120/min`. Accepts batch array or single object. Uses `UsageLog.objects.get_or_create()` and increments `time_spent_seconds`.
- `summary_view(request)`: `GET /api/summary/` returns 7-day category breakdown, today's per-site breakdown, focus efficiency, active streak, distraction list, and user settings.
- `limits_list_create_view` & `limits_detail_view`: CRUD operations for per-site limits.

- `categories_list_create_view` & `categories_detail_view`: CRUD for category overrides with retroactive updates to past `UsageLog` records.
- `sessions_view`: GET/POST for discrete focus timer records.
- `data_export_view`: `GET /api/data/export/` generates full JSON dump.
- `data_delete_view`: `POST /api/data/delete/` purges `UsageLog` and `FocusSession` records.

2.5. `dashboard/core/urls.py`

- **File Type & Layer:** Python Routing · Application URL Dispatcher
- **Responsibility:** Maps all page routes and REST API paths to view functions in `views.py`.

2.6. `dashboard/core/admin.py`

- **File Type & Layer:** Python Admin · Django Administration Panel
- **Responsibility:** Registers `UserProfile`, `UsageLog`, `UserSettings`, `SiteLimit`, `CategoryOverride`, and `FocusSession` with search fields, list filters, and column definitions for administrative inspection.

2.7. `dashboard/core/tests.py`

- **File Type & Layer:** Python TestCase · Automated Verification Suite
- **Responsibility:** Contains unit tests for domain normalization (`clean_domain`), classification rules, custom overrides, registration persona defaults, strict per-user data isolation on limits and usage logs, and token regeneration workflows.

2.8. `dashboard/core/management/commands/seed_demo.py`

- **File Type & Layer:** Python Management Command · Test Seeding Tool
- **Responsibility:** Invoked via `python manage.py seed_demo`. Creates or resets user `demo` (`demo@driftless.app` / `demo123456`) and seeds 14 days of realistic, mathematically consistent browsing logs, an active 14-day streak, custom site limits, and completed focus sessions.

2.9. `dashboard/core/templates/dashboard.html` & `auth.html`

- **File Type & Layer:** Django HTML5 Templates · Web Frontend

- **Responsibility:** Provide the user-facing web interface. `auth.html` manages onboarding and sign-in. `dashboard.html` visualizes metrics, renders Chart.js graphs, manages site limits modals, switches between Live and Demo data modes, and handles theme toggling.
